

Prescripto: Frontend Architecture Masterplan

Comprehensive UI/UX Strategy, Tech Stack, and Feature Implementation

Mission: Translate the robust Python/FastAPI backend intelligence into a premium, responsive, and highly intuitive user interface that feels like a multi-million dollar enterprise healthcare platform.

1. Executive UI/UX Vision (Avoiding the "AI-Made" Look)

To ensure Prescripto looks like a top-tier, professionally developed application, we will avoid generic templates and overly bright "default" UI frameworks. The design language will follow these core principles:

- **Design System:** Use **Shadcn UI** combined with **TailwindCSS**. Shadcn provides unstyled, highly accessible components (radix-ui) that you own and can style to look exactly like modern premium apps (e.g., Vercel, Linear).
- **Color Palette:** Trustworthy and clinical.
 - Primary: Slate Blue (#3b82f6 to #0f172a) for trust and authority.
 - Emergency: Vibrant Crimson (#ef4444) for immediate action.
 - Backgrounds: Off-white/slate (#f8fafc) to reduce eye strain, avoiding pure harsh whites.
- **Typography:** 'Inter' or 'SF Pro Display' for a clean, geometric, and highly readable interface.
- **Asset Sourcing (High-Res imagery):** Instead of AI-generated artifacts, utilize specific high-quality **Unsplash** collections. In your Next.js `next.config.js`, whitelist `images.unsplash.com` and use URLs like `https://images.unsplash.com/photo-1576091160399-112ba8d25d1d?auto=format&fit=crop&w=800&q=80` for hero banners, doctor profiles, and clinic backgrounds.

2. Modern Frontend Tech Stack

The optimal stack to interface with your FastAPI/Neon DB architecture:

- **Framework:** Next.js 14+ (App Router) for Server-Side Rendering (SEO, Speed) and API route protection.
- **Styling:** TailwindCSS & Shadcn UI.

- **State Management: Zustand** (lightweight, perfect for managing user session and active RAG chat state) and **React Query** (for caching appointments and medical records, ensuring the UI feels instant).
- **Mapping:** `react-map-gl` mapping to your Ola Maps / Mapbox APIs to render the WebSocket ambulance data.
- **WebSockets:** Native browser WebSocket API or `socket.io-client` wrapped in a React Context provider for the Phase 6 Live Telemetry.

3. Core Platform Features mapped to UI Components



One-Button Emergency Protocol

Backend Phase: Phase 3 (Routing Logic)

UI Implementation: A persistent, floating action button (FAB) or a massive central dashboard card. When pressed, a Framer Motion expanding modal appears.

- Utilizes the HTML5 Geolocation API (`navigator.geolocation.getCurrentPosition`).
- Displays a skeleton loader while fetching the FastAPI route prediction.
- Renders two massive buttons: "Book In-App Ambulance" and "Open in Uber/Ola" using standard deep-link href tags (e.g., `uber:///client_id=...&action=setPickup&dropoff[latitude]=...`).



AI Triage & Chatbot (RAG)

Backend Phase: Phase 4 (LlamaIndex & pgvector)

UI Implementation: An iMessage-style chat interface.

- **Typing Indicators:** Simulate human response time while FastAPI processes the LlamaIndex query.
- **Structured Rendering:** Because your backend returns structured JSON, do not just render text! If the backend suggests tests, the frontend maps over `response.recommended_tests` and renders them as clickable "Add to Cart" or "Book Test" cards directly in the chat stream.



Intelligent Appointment System

Backend Phase: Phase 5 (Transaction Locking & Twilio)

UI Implementation: A calendar grid using `react-day-picker`.

- **Filters:** Sidebar with sliders for Budget, Distance, and Dropdowns for Specialty.
- **Optimistic UI:** When a user clicks "Book", instantly show a success state while the backend handles the PostgreSQL lock and Twilio SMS. If the backend returns a 400 (Double Booking), gently revert the state with a toast notification.



Unified Medical Profiles

Backend Phase: Phase 2 (AWS S3 Integration)

UI Implementation: A secure dashboard split into "Patient View" and "Doctor View".

- **Drag & Drop:** Implement `react-dropzone` for seamless PDF uploads.
- **PDF Viewer:** Use `react-pdf` to display documents in-browser without forcing a download, keeping the user inside the Prescripto ecosystem.



Real-time Logistics & Bed Tracking

Backend Phase: Phase 6 (Live Telemetry & Bed Simulation)

UI Implementation: Admin/Hospital Dashboard.

- **Live Map:** Render the WebSocket stream (Phase 6) by smoothly interpolating the latitude/longitude of the ambulance icon on an Ola Map instance.
- **Bed Gauges:** Use circular progress bars (e.g., Recharts) that flash green/orange/red as the backend background task dynamically updates bed counts.

4. Proposed Value-Added Features (To increase UX & utility)

To make Prescripto stand out beyond the initial requirements, consider adding these frontend-specific features:

- **Offline-First PWA (Progressive Web App):** Configure Next.js with `next-pwa`. This allows users to "Install" Prescripto on their phone homescreen. Crucial for the Emergency Button—if the user has poor network, the app loads instantly from cache and can send coordinates via standard SMS fallback if API calls fail.
- **Family/Dependent Management:** Modify the UI to allow a master "Account" to switch profiles to children or elderly parents via a simple top-nav dropdown. Medical records and RAG context can be segmented by the active sub-profile.
- **Accessibility (a11y) Modes:** Healthcare apps must be accessible. Add a toggle for "High Contrast Mode" and "Large Text". Ensure all Shadcn components are fully screen-reader compatible (ARIA tags).
- **Vitals Dashboard:** Allow users to manually log blood pressure and sugar levels. Use `Chart.js` or `Recharts` to draw beautiful, interactive line graphs in their Unified Profile.

5. Recommended Directory Architecture (Next.js App Router)

```
prescripto-frontend/
├── app/
│   ├── (auth)/           # Login, Register pages
│   ├── (dashboard)/      # Main app wrapper (Sidebar, Nav)
│   │   ├── emergency/    # One-Button route map
│   │   ├── triage/       # RAG Chatbot UI
│   │   ├── appointments/ # Booking calendar & filters
│   │   ├── profile/      # PDF uploads & Vitals
│   │   └── logistics/    # Admin WebSocket live map
│   ├── layout.tsx        # Global providers (Zustand, React Query)
│   └── page.tsx          # Landing page
├── components/
│   ├── ui/               # Shadcn pure components (Buttons, Cards)
│   ├── maps/             # Ola/Mapbox map wrappers
│   └── chat/             # Chat bubbles, structured cards
├── lib/
│   ├── api.ts            # Axios instances connecting to FastAPI
│   ├── store.ts          # Zustand global state
│   └── websockets.ts     # Socket connection manager
└── public/
    └── assets/           # Logos, non-dynamic high-res images
```

6. High-Quality Asset Sourcing Strategy

Do not use AI-generated images for the main UI, as they can look uncanny and reduce trust in a medical app. Use these standard resources:

Asset Type	Source	Search Keywords
Hero Images / Backgrounds	Unsplash.com	"Modern hospital interior", "doctor consulting patient", "abstract blue tech"
Vector Icons (UI)	Lucide.dev (Lucide React)	N/A (Built directly into Shadcn UI)
Medical Illustrations	Freepik.com / Undraw.co	"Healthcare technology flat vector", "telemedicine illustration"